Latency Numbers Every Programmer Should Know

The table and diagram you shared are from Jeff Dean's well-known **latency comparison model**, which is extremely important in system design interviews. It helps you understand the **real-world cost difference between CPU, memory, disk, and network operations**.

1. Core Idea

Different operations in a computer have **massively different time costs**.

Even if everything looks "fast" in code, at scale:

```
None
CPU operations → nanoseconds

Memory → ~100 ns

Network → microseconds to milliseconds

Disk → milliseconds

Cross-region network → 100+ ms
```

This difference drives most system design decisions.

2. Key Latency Table

Operation name	Time
L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	100 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	10,000 ns = 10 µs
Send 2K bytes over 1 Gbps network	20,000 ns = 20 µs
Read 1 MB sequentially from memory	250,000 ns = 250 µs
Round trip within the same datacenter	500,000 ns = 500 µs
Disk seek	10,000,000 ns = 10 ms
Read 1 MB sequentially from the network	10,000,000 ns = 10 ms
Read 1 MB sequentially from disk	30,000,000 ns = 30 ms
Send packet CA (California) ->Netherlands->CA	150,000,000 ns = 150 ms

CPU & Memory Operations (Fastest)

```
L1 cache reference → ~0.5 ns

Branch mispredict → ~5 ns

L2 cache reference → ~7 ns

Mutex lock/unlock → ~100 ns

Main memory access → ~100 ns
```

These are extremely fast (CPU-level operations)

Computation & Small Processing

```
Compress 1 KB → ~10 µs
```

Compression is cheap compared to network/disk

Network Operations

```
Send 2 KB over 1 Gbps → ~20 µs

Datacenter round trip → ~500 µs
```

Network is ~1000× slower than memory

Disk & Storage

```
None

Disk seek → ~10 ms

Read 1 MB from disk → ~30 ms
```

Disk is extremely slow compared to RAM

Cross-region Network (Slowest)

```
None

CA → Netherlands → CA → ~150 ms
```

Global latency is unavoidable and expensive

3. Key Insights from the Diagram

Insight 1: Memory is fast, Disk is slow

None

```
RAM ≈ nanoseconds

Disk ≈ milliseconds
```

Disk is ~100,000× slower than memory

Insight 2: Avoid disk seeks

Random disk access is very expensive.

Better:

```
None
Sequential read > random read
```

Insight 3: Compression is cheap

```
None

Compressing data < sending over network
```

So always compress before transfer if possible.

Insight 4: Network dominates system design

```
None

Datacenter calls > local memory access
```

This is why caching + batching matters.

Insight 5: Geography matters

```
Same region → fast (~ms)

Cross-continent → slow (~100 ms+)
```

4. System Design Implications

These numbers directly explain why we use:

✔ Caching

Avoid repeated memory/disk/network access:

```
Redis instead of DB

In-memory cache instead of disk
```

✔ Message Queues

Avoid blocking network calls:

```
Async processing reduces latency pressure
```

✔ CDNs

Reduce cross-region latency:

```
None
Serve content near user
```

✔ Batch Processing

Avoid frequent expensive operations:

```
None
Process 1M events together instead of 1M requests
```

5. Mental Model (Very Important for Interviews)

Think in 4 layers:

```
None
1. CPU (ns) → fastest

2. Memory (100 ns)

3. Network (µs–ms)

4. Disk (ms) → slowest
```

6. Interview Usage Tip

If asked:

“Why caching is important?”

You can say:

```
None
Because memory access is ~100 ns while disk access is ~10–30 ms,

so caching avoids expensive disk and network operations.
```

This shows strong system intuition.

7. One-Line Summary

Latency numbers show that CPU and memory operations are extremely fast, while disk and network operations are orders of magnitude slower, which is why caching, batching, and distributed design are essential in scalable systems.